

Technical Assignment — Full-Stack Developer (.NET & Angular)

Purpose

This assignment lets us see how you design, structure, and deliver a small but complete feature across the full stack. We are not looking for a finished product — we are looking for clean, readable, well-reasoned code and correct handling of business rules.

The Scenario

Build a small internal tool called **Support Desk**, used by a support team to track customer issues from the moment they are reported until they are closed.

The interesting part of this assignment is not the CRUD — it is the **workflow rules**. Read them carefully; a large part of the evaluation is whether they are enforced correctly and in the right place in your code.

Data Model

Implement at least the following entities.

Entity	Fields
Agent	Id · Full name · Email (unique) · Department (Technical / Billing / General) · Active (true/false)
Ticket	Id · Reference — auto-generated, unique, human-readable (e.g. <code>TCK-2026-0001</code>) · Title · Description · Customer name · Customer email · Priority (Low / Normal / High / Critical) · Status (New / In Progress / Resolved / Closed) · Assigned agent (optional — a new ticket starts unassigned) · Created date · Last modified date · Resolved date · Closed date · Due date (calculated by the system, never sent by the client)
Comment	Id · Ticket · Author name · Body · Created date

A ticket has many comments. An agent can be assigned many tickets.

Business Rules

These must be enforced on the **backend**. The frontend may also reflect them for a better user experience, but the API must never rely on the client to do so.

- Due date** is derived from the priority at creation time:
 - Critical → 4 hours after creation
 - High → 1 day
 - Normal → 3 days
 - Low → 7 daysIf the priority is changed while the ticket is still open, the due date is recalculated from the **original creation date**.
- Allowed status transitions:** `New → In Progress`, `In Progress → Resolved`, `Resolved → Closed`, and `Resolved → In Progress` (reopening). Any other transition must be rejected with a clear error. In particular, a ticket cannot skip straight from `New` to `Resolved`, and a `Closed` ticket can never be reopened.
- A ticket cannot move to `In Progress` unless an **active** agent is assigned to it.
- An **inactive** agent cannot be assigned to any ticket.
- A `Closed` ticket is **read-only** — no edits, no status changes, no new comments.
- Resolved date and closed date are set by the system when the corresponding transition happens, not by the client.
- A ticket is **overdue** when its due date has passed and its status is neither `Resolved` nor `Closed`.

Backend — ASP.NET Core Web API

- Build a RESTful API using **C# and ASP.NET Core**, with **Entity Framework Core** for data access.
- Ticket endpoints:
 - List tickets with **pagination**, **search** (by reference, title, or customer name), and **filters** by status, priority, assigned agent, and overdue-only.
 - Get a single ticket including its comments.
 - Create a ticket, update its editable fields, and delete it.
 - Assign or unassign an agent.
 - Change status — a dedicated endpoint rather than a generic update is preferred; explain your choice in the README.
 - Add a comment to a ticket.
- Agent endpoints: list (with search) and get by id. Full CRUD is optional.
- Server-side **validation** with meaningful, machine-readable error responses. A rejected status transition should tell the caller *why* it was rejected.
- Consistent error handling — clients should never receive a raw stack trace.
- Use **DTOs**; do not expose EF entities directly through the API.
- Database: **SQL Server** (LocalDB is fine) or **PostgreSQL**. Include EF Core **migrations** and a **seed** dataset (roughly 5 agents and 20 tickets across different statuses and priorities, including a few overdue ones) so the app is usable immediately after setup.

Frontend — Angular

1. Ticket list page

- Table or card list showing reference, title, customer, priority, status, assigned agent, and due date.
- Overdue tickets visually highlighted.
- Search input and filters wired to the API — server-side, not client-side filtering.
- Working pagination, plus loading and empty states.

2. Ticket detail page

- Full ticket information and its comment thread, with a form to add a comment.
- Status change action that only offers the transitions currently allowed.
- Agent assignment.
- Everything disabled once the ticket is closed.

3. Create / edit ticket page or dialog

- **Reactive Forms** with client-side validation and clear validation messages.
- Server-side validation and rule violations surfaced to the user in a readable way.

4. Delete with a confirmation step.

5. HTTP calls isolated in **services** — no direct `HttpClient` usage inside components.

6. Reasonable use of **RxJS**, for example a debounced search.

Quality

1. At least **4 meaningful backend unit tests**, covering the status transition rules and the due date calculation.

2. At least **2 frontend unit tests** (Jasmine/Karma or Jest) for a service or a component.

3. A **README.md** that covers:

- How to run the backend and frontend locally, including database setup
- Where you placed the business rules and why
- Any assumptions you made
- What you would improve or add with more time
- Roughly how long the assignment took you

Submission

Send your solution as a **.zip archive** in reply to this email named `FirstName_LastName_FullStack.k.zip`. We kindly request and note that it is important that you send the completed task to **recruitment@pecb.com** and **njomza.softdev@pecb.com** within 48 hours of receiving this email